

We're so back

FYS-STK3155 - Project 1

Abdullah A. Ahmed, Anders M. Rønneberg
University of Oslo
(Dated: October 5, 2026)

Statistical learning methods such as regression analysis, resampling methods, and gradient descent are core elements of machine learning procedures. However - their accuracy do rely on a numerical limitation. We tested the methods against a fixed dataset generated from the Runge function to explore whether gradient descent-based optimization methods are able to reliably reproduce the closed form OLS/Ridge solutions and what the practical trade-off between convergence speed, learning-rate sensitivity, and computational cost are compared to the analytical solution. The results conclude that OLS minimized the test error at degree 10 with $MSE = 0.0131$, Ridge also minimizes at degree 10 through $\lambda = 10^{-6}$ with $MSE = 0.0126$. Lasso using full-batch gradient descent with Adam minimizes MSE at degree 10 with $\lambda = 10^{-4}$, where none of the coefficients were driven to exactly zero. Gradient descent reproduces the closed form solution within 10^{-8} at 117 iterations using the optimal learning rate. Our results show that the Ridge regression gave the best model for the Runge function with noise $\sigma = 0.1$ using a model of polynomial degree 10 and an L2 penalty with $\lambda = 10^{-6}$.

I. INTRODUCTION

Making predictions based on observed data is one of the core principles of machine learning. Polynomial regression with ordinary least squares (OLS) makes it easy to observe several important aspects of machine learning. In this model, complexity is controlled by a single parameter, and we control the ground truth of the generated data. Thus, we have a setup well suited for exploring the strengths and weaknesses of the model.

In this assignment, we use the Runge function, a function shaped like a steep hill, to generate evenly spaced data points with added noise:

$$f(x) = \frac{1}{1 + 25x^2}, \quad y = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2) \quad (1)$$

The Runge phenomenon, originally described for polynomial interpolation on evenly spaced points, is a well-known demonstration of how increased model complexity does not always yield a better fit [1]. Increasing the polynomial degree gives a good fit around $x = 0$ but oscillations grow towards the edges of the function $x = \pm 1$, and the error increases. This is the bias-variance trade-off: a low degree underfits and gives large bias, while a high degree overfits and gives high variance. The aim is the complexity that minimises their sum. To control complexity we add a penalty to the OLS cost function, which gives Ridge and Lasso regression. Ridge shrinks the coefficients θ towards zero and dampens the oscillations, while Lasso can set coefficients exactly to zero. The penalty strength λ and the polynomial degree must then be chosen. We use bootstrapping to estimate bias and variance as the complexity changes, and k-fold cross-validation to select λ and the degree. OLS and Ridge have closed-form solutions, but Lasso does not, so we also implement gradient descent, momentum, AdaGrad, RMSProp, Adam and stochastic gradient descent. We

test whether they reproduce the closed-form solutions, and compare convergence speed and learning rate sensitivity. The gradients are derived analytically and checked against automatic differentiation with JAX [2]. The models, resampling methods and optimizers are our own implementation, while Scikit-learn [3] is used for benchmarking and cross-validation.

Section II presents the theory upon which the methods and discussions are built upon. Section III describes the methods used to gain the results which are presented and discussed in Section IV. In Section V we conclude our results, and at last the appendix ends the document at Section VI.

The code and supporting material for this project are available on GitHub.

II. THEORY

The theory of this section follows the lecture book of Morten Hjort-Jensen [4] unless otherwise stated.

A. Regression methods

Linear regression are methods of modeling the relationship between a input and output variables through fitting a line to some data. Some set of n observations of p features representing the input data can be described as the design matrix X . The output is given by $y = f(x) + \varepsilon$ where $f(x)$ represents the true underlying function, as in our case defined in Eq. 1. For polynomial regression, as will be discussed in this report, the features are increasing powers of x : $p = \hat{y}_i = \theta_0 + \theta_1 x_i + \theta_2 x_i^2 + \dots + \theta_p x_i^p$. Written in matrix form the predicted model is

$$\tilde{y} = X\theta = \begin{pmatrix} 1 & x_1 & \cdots & x_1^p \\ 1 & x_2 & \cdots & x_2^p \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^p \end{pmatrix} \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{pmatrix} = \begin{pmatrix} \sum_{j=0}^p \theta_j x_1^j \\ \sum_{j=0}^p \theta_j x_2^j \\ \vdots \\ \sum_{j=0}^p \theta_j x_n^j \end{pmatrix} \quad (2)$$

Whether the first column of ones, the intercept, is kept, depends on the centering discussed in Section II A 3. In this report, the model is defined on the interval $[-1, 1]$. High powers of x that are of the same parity develop collinearity as the columns of X turns nearly identical and approach linear dependence.

1. The cost functions and their derivatives.

Optimizing a linear regression model depends on minimizing the error produced by the model. We quantize the error through two cost functions in this report. One is the mean squared error (MSE)

$$C_{MSE}(\theta) = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 = \frac{1}{n} \|y - X\theta\|_2^2 \quad (3)$$

and the other is the R^2 score.

$$C_{R^2}(y, \tilde{y}) = 1 - \frac{\sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2}{\sum_{i=0}^{n-1} (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=0}^{n-1} y_i \quad (4)$$

The MSE cost function finds the average of the squared differences between the observed values and the models prediction. it is an absolute measure of error in the units of y^2 which means 0 is a perfect fit. The expected test MSE cost function can never be lower than the noise variance of the data σ^2 . R^2 measures the fraction of the variation in the data that the model explains. $R^2 = 1$ means a perfect fit and $R^2 \leq 0$ means that the model is the same or worse than predicting the mean.

a. Ordinary Least Squares The three regression methods considered in this work differ only in the penalty term added to the mean squared error, and can therefore be expressed through the common cost function

$$C(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 + \lambda P(\theta) P(\theta) = \begin{cases} 0 & \text{OLS,} \\ \|\theta\|_2^2 = \sum_j \theta_j^2 & \text{Ridge,} \\ \|\theta\|_1 = \sum_j |\theta_j| & \text{Lasso} \end{cases} \quad (5)$$

where $\lambda \geq 0$ is the regularization parameter controlling the strength of the parameter. Both penalties discourage large coefficients. The Ridge penalty shrinks high coefficients towards zero whereas the Lasso penalty can set individual coefficients exactly to zero. They both introduce some amount of bias in order to reduce variance

significantly. Deriving $C(\theta)$ gives the gradient of the cost function

$$\nabla_{\theta} C(\theta) = \frac{2}{n} X^T (X\theta - y) + \begin{cases} 0 & \text{OLS,} \\ 2\lambda\theta & \text{Ridge,} \\ \lambda \text{sgn}(\theta) & \text{Lasso} \end{cases} \quad (6)$$

The Lasso expression is valid only for $\theta \neq 0$ since $|\theta|$ is not differentiable at zero. For OLS and Ridge, the gradient is linear in θ , and setting it to zero gives the normal equations

$$\begin{aligned} (X^T X + n\lambda I)\theta &= X^T y \\ \Downarrow \\ \hat{\theta} &= (X^T X + n\lambda I)^{-1} X^T y \end{aligned} \quad (7)$$

where the latter is the closed form solution, and where $\lambda = 0$ describes OLS. For Lasso, the term $\text{sgn}(\theta)$ is nonlinear and not continuous, meaning no closed form solution exists, and the parameters must be found iteratively. The penalty term is clearer when written as a singular value decomposition (SVD) $X = U\Sigma V^T$. Which gives the normal equations on SVD form:

$$\hat{\theta}_{OLS} = V\Sigma^{-1}U^T y \quad (8)$$

$$\hat{\theta}_{Ridge} = V(\Sigma^2 + n\lambda I)^{-1} \Sigma U^T y \quad (9)$$

Where $X \in \mathbb{R}^{n \times p}$, $U \in \mathbb{R}^{n \times n}$, $\Sigma \in \mathbb{R}^{p \times p}$ and $V \in \mathbb{R}^{p \times p}$ are matrices with columns u_i and v_i respectively. The Ridge prediction then becomes

$$X\hat{\theta} = \sum_j u_j \frac{\sigma_j^2}{\sigma_j^2 + n\lambda} u_j^T y \quad (10)$$

Colinear combinations appear as modes with small σ_j . These are most affected by the penalty, while modes of larger values of σ_j are barely shrunk.

The second derivative of the cost function gives what is known as the Hessian function, which at vector form is:

$$H = \frac{2}{n} X^T X + 2\lambda I \quad (11)$$

where $\lambda = 0$ gives the hessian matrix of OLS. $X^T X$ has no negative eigenvalues, which means that the cost function has the shape of a convex bowl. In the point where the gradient is zero, we find the functions global minimum. In the case of OLS, collinearity of high powers of x could develop eigenvalues of $X^T X$ that are zero or extremely close to zero. For these the inverse does not exist or are numerically unreliable and the SVD are needed to produce the inverse. The Ridge term raises every eigenvalue by $2\lambda I > 0$ ensuring that the matrix is always invertible.

2. Conventions in scaling of the cost function

`Scikit-Learn` [3] scales the squared error by $1/2n$ for lasso and does not scale it at all for Ridge whereas Eq. 5 uses $1/n$. Rescaling the cost does not move its minimum, so `scikit-learn`'s penalty parameter α relates to ours as $\lambda = 2\alpha$ for Lasso and $\alpha = n\lambda$ for Ridge. We set `fit_intercept = False` in computing the comparisons, since the data are already centered. The $\lambda\|\theta\|_1$ term is not differentiable at $\theta_j = 0$, so we use the subgradient there. For $|\theta_j|$ the subdifferential is $\text{sgn}(\theta_j)$ when $\theta_j \neq 0$ and the whole interval $[-1, 1]$ when $\theta_j = 0$. Automatic differentiation (Section IID 1) with `JAX` [2] returns 1.0 at $\theta_j = 0$ while our own implementation uses 0.

3. Scaling

The penalties of Ridge and Lasso both treat all coefficients the same way. This becomes problematic when they are of different scales. When the data consists of numbers of different units, the fitted coefficients differ in enormous magnitude from the coefficients describing a similar feature with different unit. To solve this difference we introduce a scaling of the data before the Ridge or Lasso penalty is applied. For each feature j in the design matrix X the mean and the standard deviation is computed:

$$\mu_j = \frac{1}{n} \sum_{i=0}^{n-1} X_{ij}, \quad s_j = \sqrt{\frac{1}{n} \sum_{i=0}^{n-1} (X_{ij} - \mu_j)^2} \quad (12)$$

and scaled as

$$X^{\text{scaled}} = \frac{X_{ij} - \mu_j}{s_j} \quad (13)$$

The target values will also go through a process of centering. This is done by subtracting the mean of the training targets: $y_i^{\text{centered}} = y_i - \bar{y}_{\text{train}}$. This removes the necessity of the intercept column when estimating the parameters. Instead the intercept is accounted for by adding the target mean back to the predictions as such $\hat{y} = X^{\text{scaled}}\theta + \bar{y}_{\text{train}}$. Standardizing the features makes all columns equally large, so the cost function is steep in any direction, and it is easier to choose a stable learning rate. However, the polynomial features are still correlated with each other. This keeps the cost function almost flat in some directions, so the Hessian is still ill-conditioned and gradient descent still converges slowly along them.

B. The Bias-Variance Tradeoff

Another way to understand how MSE changes with complexity of the model is with a statistical approach, the so called Bias-Variance tradeoff. Figure 1 is a simple

model of this tradeoff. Bias is the error that comes from a model's inability to accurately fit the true underlying function of the data. It reflects how much the model's predictions systematically differ from the true function, and it does not include the irreducible noise σ^2 .

Variance measures how sensitive a model is to small changes in the training data. A high-complexity model tends to have higher variance because its fitted function can change significantly when the training data changes slightly.

To derive the Bias-Variance tradeoff, we try to decompose the MSE into variance and bias. Assume the data are generated by

$$y = f(\mathbf{x}) + \varepsilon, \quad \mathbb{E}[\varepsilon] = 0, \quad \text{Var}[\varepsilon] = \sigma^2, \quad (14)$$

where f is a deterministic function and the noise ε is independent of the model prediction $\tilde{y}$ (the model is trained on data that does not include the noise of the new point). For readability we ignore the dependence on $\mathbf{x}$. Expanding the square and using linearity of the expectation gives

$$\mathbb{E}[(y - \tilde{y})^2] = \mathbb{E}[y^2] - 2\mathbb{E}[y\tilde{y}] + \mathbb{E}[\tilde{y}^2]. \quad (15)$$

We treat the three terms separately.

Term 1. Since f is deterministic and $\mathbb{E}[\varepsilon] = 0$,

$$\mathbb{E}[y^2] = \mathbb{E}[(f + \varepsilon)^2] = f^2 + 2f\mathbb{E}[\varepsilon] + \mathbb{E}[\varepsilon^2] = f^2 + \sigma^2. \quad (16)$$

Term 2. By independence of ε and $\tilde{y}$,

$$\mathbb{E}[y\tilde{y}] = \mathbb{E}[(f + \varepsilon)\tilde{y}] = f\mathbb{E}[\tilde{y}] + \mathbb{E}[\varepsilon]\mathbb{E}[\tilde{y}] = f\mathbb{E}[\tilde{y}]. \quad (17)$$

Term 3. Adding and subtracting $\mathbb{E}[\tilde{y}]^2$ and using the definition of the variance,

$$\mathbb{E}[\tilde{y}^2] = \mathbb{E}[\tilde{y}^2] - \mathbb{E}[\tilde{y}]^2 + \mathbb{E}[\tilde{y}]^2 = \text{Var}[\tilde{y}] + \mathbb{E}[\tilde{y}]^2. \quad (18)$$

Inserting the three results into (15) and completing the square,

$$\begin{aligned} \mathbb{E}[(y - \tilde{y})^2] &= f^2 + \sigma^2 - 2f\mathbb{E}[\tilde{y}] + \text{Var}[\tilde{y}] + \mathbb{E}[\tilde{y}]^2 \\ &= (f - \mathbb{E}[\tilde{y}])^2 + \text{Var}[\tilde{y}] + \sigma^2 \\ &= \text{Bias}^2[\tilde{y}] + \text{Var}[\tilde{y}] + \sigma^2, \end{aligned} \quad (19)$$

where $\text{Bias}[\tilde{y}] = \mathbb{E}[\tilde{y}] - f$.

As shown, the expected test MSE decomposes into squared bias, variance and the irreducible noise, and it can be estimated by averaging over bootstrap resamples of the training set and over the test points.

C. Resampling techniques

Resampling techniques estimate how well a model generalizes to new data by repeatedly fitting it to different subsets of the available data, which gives a more reliable

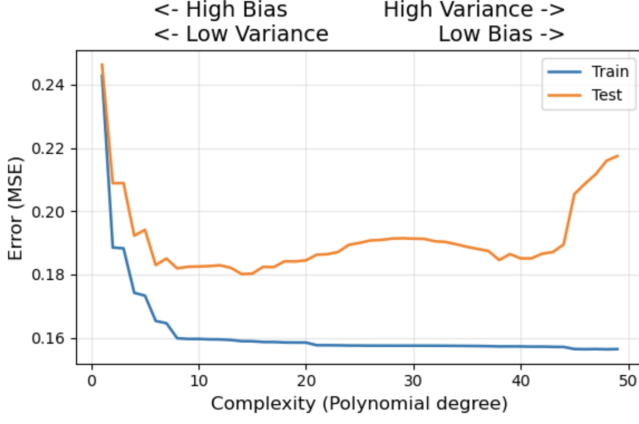


Figure 1: Error as function of the complexity (degrees 1 to 49) using ridge regression to showcase a simple model of the bias-variance tradeoff. datapoint $n=1000$ with 60/40 training/test split, noise $\sigma=0.4$ and $\lambda=0.0$

estimate of the test error than a single train/split test. One quality measure of the model splits the data into two parts. One training part, that is used to fit the model, and one smaller fraction used to test the model. When evaluating the model on data it has not been fitted to we are allowed to view the variance of the model. The training error reflects bias, but hides variance, since a flexible model adapts to noise.

Bootstrap is a method that generates new datasets through randomly resampling the original data. When bootstrapping, resampling is allowed, which opens for adding the same data several times in the same bootstrap dataset, while other may not appear at all. We then produce models from all the bootstrapped datasets and make predictions. Each bootstrapped model is a little different, resulting in different predictions. From this the expectation value $\mathbb{E}[\tilde{y}]$ can be calculated through the average over all bootstrap predictions. Through this one can find reliable values for the variation of the model, as well as MSE and bias.

In order to reduce the dependence of the points that happen to end up in the test set and their effects in the test error, cross validation (CV) is applied. Here the data is divided into k equally sized folds. The model is fitted k times, each on $k-1$ folds and tested on the remaining one. MSE is calculated for each test fold and the average MSE over all folds is used to describe model performance.

1. Avoiding data leakage

To avoid data leakage in the resampling methods, it is important to make sure that scaling and centring do not occur before the data is split. We split the data first, find the mean and standard deviation of the training data, and use these to scale both the training and the test data. If we scale before splitting, the training data will contain

information from the test data, since the test data points were used to calculate the mean and standard deviation.

D. Gradient Descent

The idea of gradient descent (GD) is finding the global minimum of a function $F(x)$ through moving along the negative gradient $-\nabla F(x)$. In this report we wish to minimize the cost function (Eq. 5). The Hessian of the cost function (Eq. 11) is positive definite and does not depend on θ . Because of this we know that the cost function has one global minimum and has the same curvature everywhere. This allows the iterative process of GD

$$\theta_{k+1} = \theta_k - \gamma \nabla_{\theta} C(\theta_k). \quad (20)$$

to reach the same global minimum as the closed form solution. GD continuously updates the parameters with a fixed learning rate γ . We define the error $e_k = \theta_k - \theta^*$, where θ^* is the minimum. For a quadratic cost $\nabla C(\theta) = He$ the iteration can be written

$$e_{k+1} = (I - \gamma H)e_k \quad (21)$$

Through diagonalisation, spectral decomposition where $H = Q\Lambda Q^T$ has orthonormal eigenvectors and eigenvalues $\mu_i > 0$ and the error of the eigen basis $\tilde{e} = Q^T e$ gives

$$\tilde{e}_{k,i} = (1 - \gamma\mu_i)^k \tilde{e}_{0,i} \quad (22)$$

This equation illustrates how error is scaled by the factor $|1 - \gamma\mu_i|$ for every step. When this factor is greater than 1, the iteration diverges in that direction i . For all directions to converge one can define

$$0 < \gamma < \frac{2}{\mu_{max}} \quad (23)$$

Where μ_{max} is the maximal eigenvalue of the Hessian matrix. This is the stability limit. Exceeding this will result in divergence or oscillation.

For speed we must evaluate the slowest direction, knowing that every component shrinks at least as fast as this one. We define the worst contraction factor $\rho(\gamma) = \max_i |1 - \gamma\mu_i|$. Knowing that this is a straight line through the interval, and that the largest absolute values are at these endpoints, we turn to these two significant eigenvalues $\rho(\gamma) = \max(|1 - \gamma\mu_{min}|, |1 - \gamma\mu_{max}|)$. In order to find the optimal γ we find the crossing point of the two terms of $\rho(\gamma)$

$$1 - \gamma\mu_{min} = \gamma\mu_{max} - 1 \Rightarrow \gamma_{opt} = \frac{2}{\mu_{max} + \mu_{min}} \quad (24)$$

Fitting this optimal learning rate in $\rho(\gamma)$ gives the contraction rate

$$\rho(\gamma_{opt}) = \frac{\mu_{max} - \mu_{min}}{\mu_{max} + \mu_{min}} = \frac{\kappa - 1}{\kappa + 1}, \quad \kappa = \frac{\mu_{max}}{\mu_{min}} \quad (25)$$

where κ is the condition number of the Hessian matrix. Through equations its evident that the best achievable learning rate derives from the Hessian eigenvalues.

Several variants of GD have been developed to address its shortcomings. The momentum method takes previous steps into account instead of stepping only according to the current gradient. AdaGrad gives each parameter its own learning rate, scaled by its accumulated squared gradients. This comes with the risk of stalling, as the learning rate continuously shrinks. RMSProp uses a moving average of the squared gradients rather than the full sum used in AdaGrad, which reduces the risk of stalling. The Adam method combines momentum with RMSProp’s scaling and adds a bias correction to its moment estimates. This powerful combination has made it a common default choice.

When the number of features p is large, iterative optimizers are preferred over the analytical approach. Forming and inverting the $p \times p$ matrix $X^T X$ then becomes expensive, with a cost that grows as $O(p^3)$. For the moderate polynomial degrees used here, p is small and the closed-form solution remains cheap. For datasets with many data points, it is also common practice to compute the gradient on small random subsets of the data. This is called stochastic gradient descent (SGD). Strictly speaking, SGD uses a single data point per step and mini-batch SGD uses small batches, but the term SGD is commonly used for both. SGD can be combined with all of the methods mentioned above.

1. Automatic differentiation

Automatic differentiation is one way to calculate derivatives, by applying the chain rule through the sequence of elementary operations that a program performs, so that in the end we only need the derivatives of simple known functions. It is exact up to rounding errors, but unlike symbolic differentiation it does not manipulate formulas to produce an expression for the derivative, and unlike numerical differentiation it does not estimate the derivative with, for example, finite differences. Automatic differentiation can be done in forward or reverse mode. For a cost function with many parameters and a single scalar output, reverse mode computes all partial derivatives in roughly the cost of one function evaluation, which makes it much cheaper than forward mode. We use it through JAX as an independent check of our hand-written gradient.

III. METHODS

A. Fitting the models

To obtain a good model for the Runge function, we need methods both for fitting models and for validating which of them works best. We fit polynomial models with Ordinary Least Squares (OLS), Ridge regression and Lasso regression. All three minimize the cost function in Eq. (5), which is the mean squared error (MSE) plus a penalty on the parameters. OLS and Ridge have analytical closed-form solutions, which are cheap to compute. Lasso has no closed form, since the L_1 penalty is not differentiable at zero, so it must be solved iteratively with GD. GD can converge slowly for high-degree polynomials, since the cost function can have some very flat directions. To speed this up we use momentum and the adaptive methods AdaGrad, RMSProp and Adam, which adapt the step size for each parameter. We also experiment with stochastic gradient descent (SGD), where each update uses only a mini-batch of the data, which makes every update cheaper.

B. Comparing the GD methods

For OLS and Ridge we measure how close each gradient descent method gets to the closed-form solution. Once a method is close enough to the closed-form solution (within a tolerance of 10^{-8}), we stop iterating and compare how many iterations each method needs. Lasso has no closed form, so we instead compare the methods by the value of the cost function as a function of the iteration. It is also harder to decide when to stop iterating, since there is no analytical solution to compare with. Because the Lasso cost is convex, all methods approach the same minimum, so a method that reaches a lower cost, and reaches it faster, is the better optimizer.

For SGD we vary the batch size, the number of epochs and the learning-rate schedule $\gamma_t = t_0/(t + t_1)$, where t is the update number. The methods are compared by the number of single-point gradient evaluations, so one epoch always costs n evaluations.

C. Model validation

To determine how good a model is we use training and test data. The fitting of the models will be done on the training data, and the MSE on the test data can be used to evaluate the strength of the model. The model with the lowest MSE will be the strongest. In this report we use both a single split with 3/5 of the data for training and 2/5 for testing, and k -fold cross-validation with $k = 5$ and $k = 10$. We use k -fold cross-validation to estimate the best polynomial degree and optimal λ for fitting the Runge data.

To numerically verify the bias-variance trade-off in Eq. (19) we use the bootstrap method. For each polynomial degree we fit $B = 50$ models on resampled training sets and predict on the test set. The variance is the spread of the predictions across resamples, and the bias is estimated from their mean. Since we compare the predictions with the noisy test targets $y = f(x) + \varepsilon$ instead of the noise-free $f(x)$, the estimated Bias² absorbs the noise variance σ^2 .

D. Data and preprocessing

We generate data from the Runge function Eq. (1) at $n = 100$ equally spaced points in $[-1, 1]$, and add Gaussian noise with standard deviation $\sigma = 0.1$. The features are standardized because of what was mentioned in Section II A 3 with the mean and standard deviation of the training data, and the target is centred with the training mean. Models are evaluated with the mean squared error (MSE) and R^2 .

The scaling of the data when used for bootstrapping and k-fold cross-validation are done as described in Section II C 1.

Other values of n and σ used in specific experiments are stated where they appear.

E. Implementation

The code is written in Python with NumPy, JAX (automatic differentiation) and scikit-learn. All code and results are available at GitHub.

Most of the code implements and verifies the building blocks: the closed-form solutions, the gradient descent methods, k-fold cross-validation and the bootstrap. The main result, Table I, is generated by a function that loops over polynomial degrees 1–15 and, for Ridge and Lasso, over 13 logarithmically spaced values of λ from 10^{-6} to 1. For each combination it computes the mean validation MSE with 5-fold cross-validation, and the table lists the best λ for each degree and method. OLS and Ridge are fitted with the closed form. Lasso is fitted with Adam ($\gamma = 0.01$, 3000 iterations starting from $\theta = 0$), using a fixed number of iterations. Adam was chosen as the gradient descent method since it consistently gave good results and rarely diverged compared to for example momentum.

F. Implementation validation

We verified the implementation in four ways. First, gradient descent with our hand-written gradient and with the JAX gradient gives final θ that differ by less than 10^{-15} . This can be observed in Figure 2 where GD and GD_{Jax} overlap perfectly.

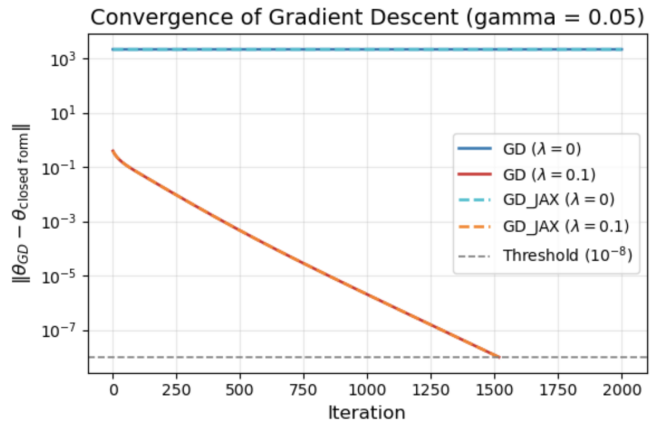


Figure 2: the distance between θ_{GD} and θ_{OLS} as a function of the gradient descent iteration. the 4 plots are using $\lambda = 0$ and 0.1 for both selfmade and JAX gradient descent. $\gamma = 0.05$ for all plots ($n = 100$ points)

Second, gradient descent for Ridge reproduces the closed-form solution to within 10^{-8} . Third, our own cross-validation agrees with scikit-learn’s KFold cross-validation as shown in Figure 3.

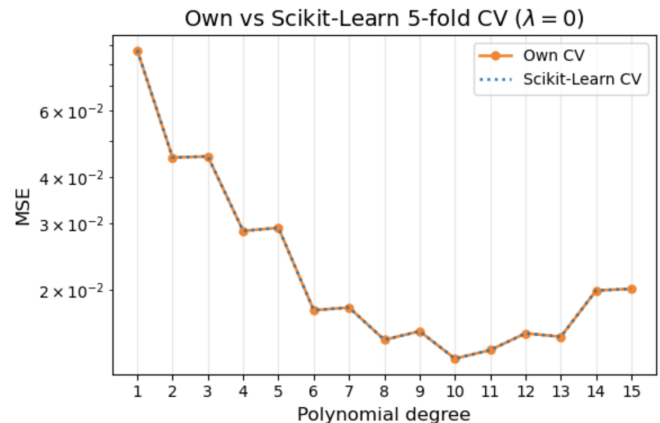


Figure 3: Average MSE of self made Cross validation function against Scikit-learn CV for degrees up to 15 with $k = 5$ $\lambda = 0$ ($n = 100$ points)

Fourth, for Lasso at degree 2 and 4 and $\lambda = 0.01$, the cost reached by Momentum and Adam agrees with scikit-learn’s Lasso to better than 10^{-12} .

G. Use of AI tools

AI, specifically Claude Sonnet 5.5 was used to turn handwritten notes into latex format, take data generated from self-written code into presentable plots and tables, fix hard-to-detect bugs in the code, as well as for checking spelling or logical errors in the report. AI was also used to help with structure and making sure theory/statements

are correct.

IV. RESULTS AND DISCUSSION

A. OLS and Ridge

Figure 4 shows the training and test MSE and R^2 as functions of polynomial degree for OLS $\lambda = 0$ and for Ridge regression with $\lambda = 0.01, 0.1$ and 1 . In the OLS panel, the MSE and R^2 increase and decrease respectively with a significant amount for higher levels of complexity for test data, while training data steadily declines. The reason are likely to derive from the model fitting to the noise of the dataset, resulting in overfitting. If we consider the training data across all the panels, a trend of increasing MSE becomes evident. This is in line with theory for Ridge (Section II A 1 a), which states that in order to reduce variance it introduces some bias. The penalty stops the model from fitting the training data perfectly, introducing a systematic error. The test data of the model with $\lambda = 0.1$ shows the characteristic shape for Bias-Variance trade-off described in section II B. This phenomenon will be discussed later in this section (Sec. IV B. A further illustration of the effects of the Ridge penalty can be seen at high degrees. At $\lambda = 0$ the model struggles with high variance, which disappears already at $\lambda = 0.01$. All panes show underfitting at low degrees. The figure also shows the limitations of this analysis. With a 40/60 test-train-split of 100 datapoints, the test set contains only 40 points, and the test error depends on which points happen to be selected. This could explain the growth in MSE for OLS at degrees 8 and 9. For Ridge with $\lambda = 1$, the test metrics are far better than the training metrics, which could suggest that this test set by chance contains few points near the sharp peak at $x = 0$. The figures therefore illustrate that model complexity and the penalty λ shift the balance between bias and variance, but a single train/test split is not sufficient to quantify this balance or to select the best model. In the next section (Sec. IV B, the bias-variance trade-off will therefore be studied explicitly with the bootstrap, and the model selection is revisited with cross-validation.

Figure 5 illustrates the effect of Ridge regression on coefficients. The OLS coefficients, shown on a logarithmic scale, grow from order one below degree 6 to several hundred at degrees 14–15, with alternating signs. This indicates multicollinearity - high similarity between the variables of the model. On $[-1, 1]$ the high-degree terms are nearly linearly dependent, so large contributions of opposite sign almost cancel on the training points. This makes the coefficients, and hence the predictions near the endpoints, very sensitive to noise. Reducing this effect is the main goal of implementing the Ridge penalty. The Ridge coefficients stay below order 1 in magnitude and gathers closer to 0 as the degree increases.

The shrinkage behavior in figure 5 can be understood

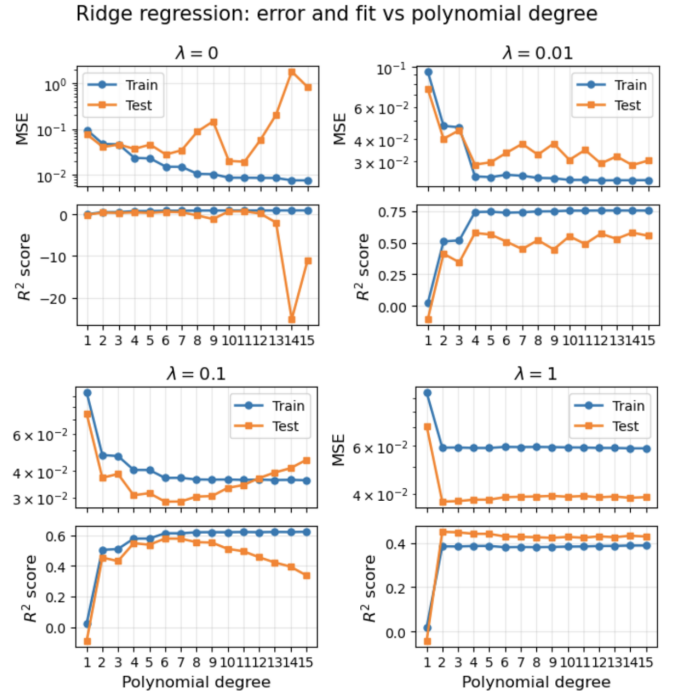


Figure 4: training and test MSE and R^2 for OLS and Ridge fits of polynomial degree 1-15. Top: MSE; Bottom: R^2 . Training data in blue, test data in orange. $n = 100$ points, $\sigma = 0.1$ 60/40 train/test split.

through the SVD (Section II A 1 a). The SVD splits the problem into independent directions, each with a singular value σ_j that tells how much information the data contain in that direction. For high polynomial degrees the columns of X are nearly identical, so some directions have very small σ_j . In Eq. 8, the OLS solution divides by σ_j , so noise along these directions is blown up. This explains the large, alternating OLS coefficients at high degree. Eq. 9 adds $n\lambda$ to σ^2 in the denominator, so the division can no longer blow up. Directions with large σ_j are barely affected, while directions with small σ_j are suppressed, as seen in Eq. 10. Since the unstable directions have very small σ_j , even a small λ removes them.

B. Bias-Variance, Bootstrapping and Cross Validation

Figure 6 demonstrates the characteristics of the bias-variance tradeoff described in Section II B, using the bootstrap to estimate the bias and variance terms. It shows how the two terms contribute to the test error as the model complexity increases. For degrees 1–4, the bias dominates, while the variance is of the order 10^{-3} to 10^{-2} . In this region, every bootstrap fit is nearly identical, though all of them underfit the Runge function due to their simplicity. Degrees 5–8 form a crossover interval, where the variance increases rapidly and both terms contribute significantly to the error. Here the two terms are

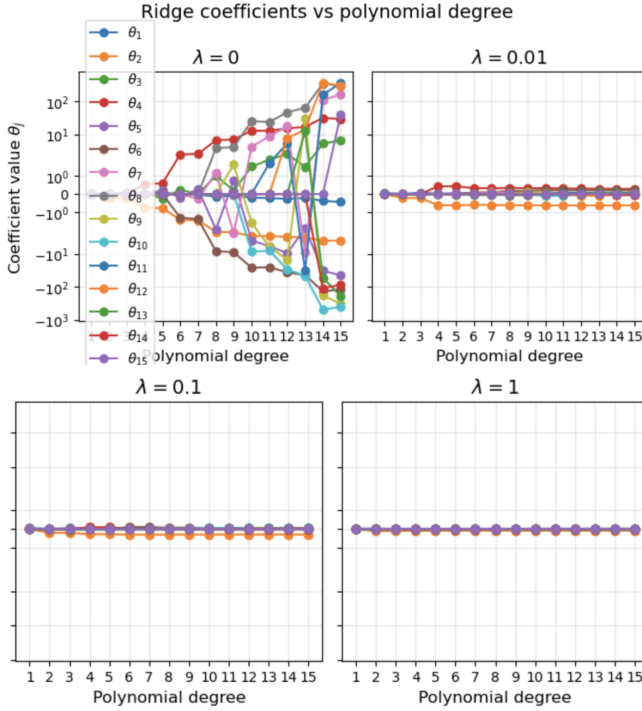


Figure 5: Thetas as function of polynomial degrees 1-15. model trains on $n=60$ points and the 4 plots show the results using different λ s for the ridge regression.

balanced, and the test error reaches its minimum of 0.04 at degree 4, which is the optimal complexity in this case. From degree 9 to 15, the variance dominates. It increases by several orders of magnitude, from about 1 to about 10^3 , and the test error follows it closely. Each bootstrap sample now produces a very different polynomial, which is a clear sign of overfitting. At high degrees, the bias also increases and makes up a significant part of the total error. Bias should not increase with model complexity, and one can argue what we see is not real bias. It is an artifact of the method, with two causes. First, all bootstrap samples are drawn from the same fixed training set, so an error caused by fitting the noise in that particular set is shared by all fits and appears as bias rather than variance. Second, the average prediction is computed from a finite number of bootstrap samples, and is therefore itself uncertain. Its squared error is roughly the variance divided by the number of samples, and this contribution is counted as bias. When the variance is very large, this leakage dominates the measured bias.

Figure 7 shows how the error depends on the amount of data, by plotting the bias, variance and test error against the number of data points. In this graph, the effect of data point on variance is in focus, as the degree is fixed at 15. For small n , the variance dominates, with the bias a few orders of magnitude lower. As n increases, the variance decreases steadily, while the bias and the total error level off and approach 10^{-2} at around 400 data points. The true bias does not depend on n , and

for a degree-15 polynomial it is small compared with the noise. The decrease in measured bias at small n is caused by the same leakage from the variance as in Figure 6, which vanishes as the variance decreases. Because the noisy data y is used instead of the true function f (Eq. 1) in the bias term, the measured bias is the sum of the true bias and the noise variance. For large n , the measured bias is therefore dominated by the noise term $\sigma^2 = (0.1)^2 = 10^{-2}$, which is the lower limit for the error of any model on this data.

In the methods above, the models are evaluated on a single, fixed test set. As discussed in Section II C, the results depend on which data points end up in the test set, especially when n is small. Cross-validation (Section II C) reduces this dependence: every data point is used for testing exactly once, and the test error is averaged over all folds, which gives a more reliable estimate. Figures 8 and 9 show the cross-validated MSE as a function of polynomial degree for $k = 5$ and $k = 10$ folds, respectively, with plotted with standard deviations.

For OLS ($\lambda = 0$), the MSE curve has a valley-like shape with its minimum at degree = 10 for both values of k . The minimum mean value, $\text{MSE} = 0.027$, is close to the lower limit set by the noise variance, $\sigma^2 = 10^{-2}$. The shape is another illustration of the bias-variance trade-off: bias dominates at low degrees and variance at high degrees. The increase in variance is also visible in the error bands, which widen at high degrees. A high-degree polynomial changes considerably depending on which points it is trained on, so the MSE varies strongly from fold to fold.

When the Ridge penalty is applied, the curves flatten into a plateau instead of rising at high degrees. The penalty prevents overfitting, but at the cost of added bias. This is why every Ridge plateau lies above the minimum of the OLS curve, and why the plateau rises with increasing λ . For the values of λ tested here, Ridge therefore introduces a significant amount of bias. The variance, however, is kept under control, as the constant width of the error bands shows.

These properties of OLS and Ridge do not depend on k , as the similarity between figures 8 and 9 shows. The optimal polynomial degree and the ranking of the λ values are the same in both figures, which indicates that the model selection is robust to the choice of the number of folds. The figures differ, however, in the width of the error bands, which is a consequence of the fold size. For $k = 5$, the 100 data points are divided into five test folds of 20 points each; for $k = 10$, each test fold contains only 10 points. The MSE of a smaller fold is more sensitive to individual points, so the spread between the fold scores increases.

Comparing with the bootstrap results in Figure 6 shows that the two methods point to very different optimal complexities. With the bootstrap, degrees 2 to 6 give the lowest test errors, whereas cross-validated OLS performs best for degrees 10 to 13. For the highest degrees, the bootstrap test error reaches 10^2 to 10^3 , far above the

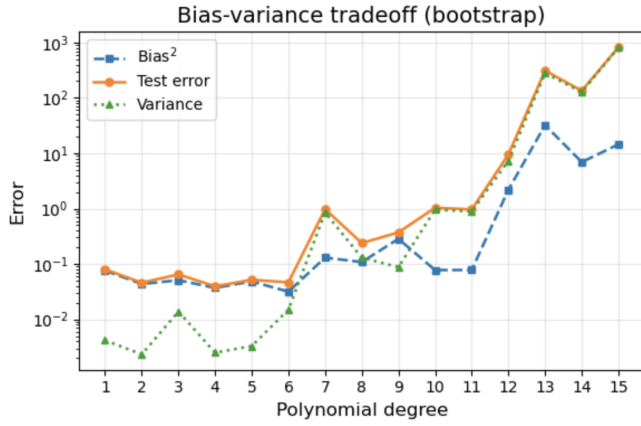


Figure 6: Error as function of polynomial degree (complexity) using $n=60$ points, $\text{bootstrap}=50$ and $\lambda=0.0$. This figure showcases the bias-variance-tradeoff when increasing complexity of the model.

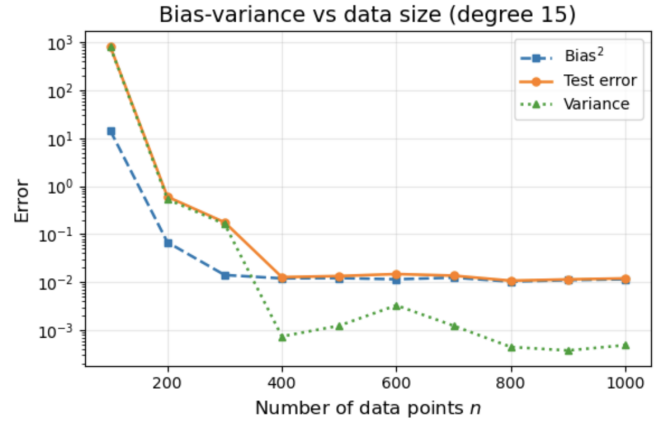


Figure 7: Error as function of number of data points using $\text{degree}=15$, $\text{bootstrap}=50$ and $\lambda=0.0$. This figure showcases the bias-variance-tradeoff when increasing amount of points in the data.

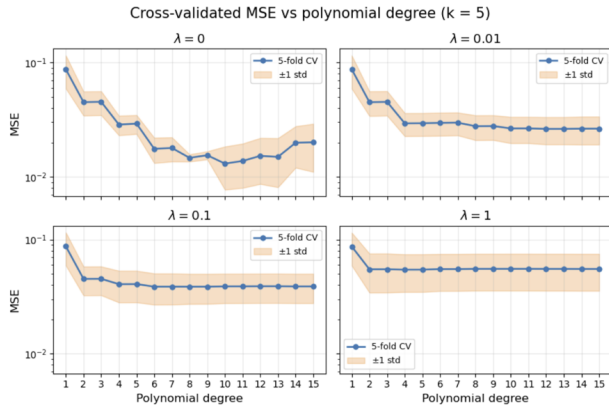


Figure 8: Average MSE against polynomial degree using 5-fold cross-validation with $n=100$ points. The different plots differ by their λ and they also show the STD at each degree.

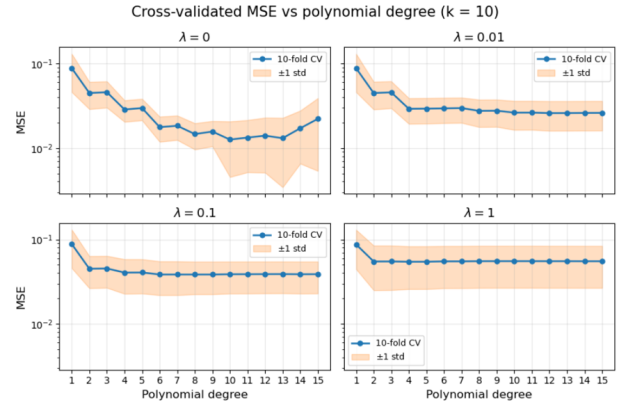


Figure 9: Average MSE against polynomial degree using 10-fold cross-validation with $n=100$ points. The different plots differ by their λ and they also show the STD at each degree.

largest cross-validated MSE. These values are extreme, given that the function itself only takes values in $[0, 1]$. A bootstrap sample contains on average only about 63% unique data points [5], so some samples will lack points near the edges of the interval. Such errors get evened out through cross-validation. High-degree polynomials fitted to such samples must then extrapolate towards the edges, where they diverge badly. In addition, the bias increases with degree. This contradicts the expected behavior. It can be explained by the same mechanism: the bias is computed from the average prediction over all bootstrap samples, and a handful of very poor fits can pull this average far from the data.

C. Gradient Descent

For OLS and Ridge regression the optimal parameters $\hat{\theta}$ have a closed-form solution, while Lasso does not. We therefore test our gradient descent (GD) code on OLS and Ridge first, where the exact answer is known.

Figure 10 shows the distance $\|\theta_{GD} - \hat{\theta}\|_2$ between the GD parameters and the closed-form Ridge solution as a function of the number of iterations, for six learning rates. Runs stop when this distance falls below 10^{-8} marked in the graph as a dashed line. All convergent curves are straight lines on the logarithmic axis, meaning that the error is reduced by a constant factor in every iteration. For the three hand-picked learning rates, $\gamma = 0.01, 0.05$ and 0.1 , a larger step gives faster convergence. With $\gamma = 0.1$ the threshold is reached after 755 iterations. $\gamma = 0.05$ and $\gamma = 0.01$ did not converge at

1000 iterations.

The remaining three learning rates are calculated from the eigenvalues of the Hessian. Equation 24 gives the fastest learning rate, which we tested against one rate just above and one rate just below. In the figure, we tested the theoretically optimal rate γ_{opt} together with one learning rate just above and one just below the stability limit : $\gamma_{conv} = 0.98 \gamma_{max}$ and $\gamma_{div} = 1.02 \gamma_{max}$. As predicted, $\gamma_{opt} = 0.667$ (red) converges fastest, reaching the threshold after 117 iterations. The increased learning rate value of $\gamma_{conv} = 0.7$, does not result in fewer iterations even though the value is higher yet. Rather, it reaches the lower limit at 409 iterations. The reason lies in the steepest direction of the cost function, the eigenvalue μ_{max} . Inserting γ_{conv} in the factor of Eq. 22 scales the error by

$$1 - \gamma_{max} \mu_{max} = 1 - 0.98 \cdot 2 = -0.96$$

per iteration, independent of the data. In this direction every step overshoots the minimum: the error changes sign, landing on the opposite side, and shrinks by only 4%. At $1.02 \gamma_{max}$ (brown) the same factor becomes $1 - 1.02 \cdot 2 = -1.04$. The overshoot is now larger than the distance to the minimum, so the error grows by 4% each step resulting in a divergence. Increasing the learning rate is therefore only helping the number of iterations up to the point of γ_{opt} . Below this point the flattest direction limits the speed, above it the algorithm overshoots, in the point the two are balanced.

Figure 11 shows the parameter error for the five update rules with $\gamma = 0.1$ and $\lambda = 0.1$. For plain gradient descent, convergence require $\gamma < 2/\mu_{max}$. Since μ_{max} grows with polynomial degree, plain GD converges at degree 2 but diverges at higher degrees. Momentum diverges together with plain GD from degree 6. AdaGrad never diverges, since its accumulated squared gradients only reduce the step size, but it converges slowly and had not reached the tolerance within 1000 iterations from degree 6 and up. RMSProp does only converge to the tolerance at degree 2. Beyond this its moving average keeps the step size from vanishing, so it oscillates around the solution. Adam, which combines momentum with RMSProp-type scaling, was the only method reaching the tolerance before 1000 iterations at degree 6.

D. Stochastic gradient descent

Since Lasso must be solved iteratively, we test whether stochastic gradient descent (SGD) can give equally good parameters at a lower cost per update. We test it on Ridge regression, where the closed-form solution is known and the distance to it can be measured, and compare the methods by the number of single-point gradient evaluations.

Figure 12 shows that small batches make progress fastest at the start: with $M = 20$ the error is around

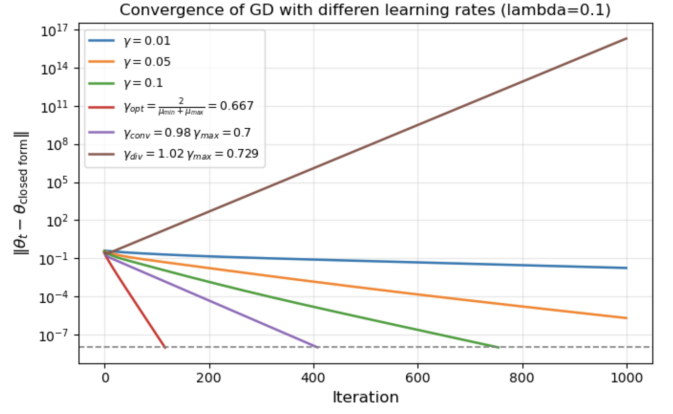


Figure 10: Distance between θ_{GD} and $\theta_{closed-form}$ as function of iterations using plain gradient descent (degree = 15). The different graphs show different learning rates and the gray striped line is the convergence threshold (10^{-8}) ($n = 100$ points).

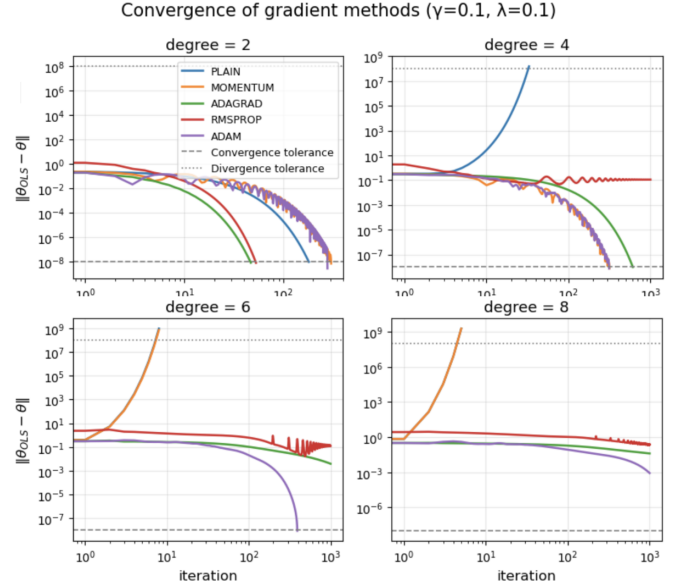


Figure 11: Distance between θ_{GD} and $\theta_{closed-form}$ using plain GD, momentum and the 3 adaptive methods. The divergence threshold is 10^8 and the convergence threshold is 10^{-8} ($n = 60$ points).

10^{-2} after a single epoch (600 evaluations), while full-batch gradient descent ($M = 600$) takes around 20000 evaluations to reach the same error. With a fixed learning rate the stochastic versions then stop improving and fluctuate around a noise level that grows as the batch gets smaller ($M = 1$ stays around 10^{-1}). The reason is that a mini-batch gradient is noisy even at the minimum, where the full gradient is zero. The full-batch method improves steadily and gets below the error of $M = 20$ after around 22000 evaluations.

The noise of SGD is suppressed by a decaying learning

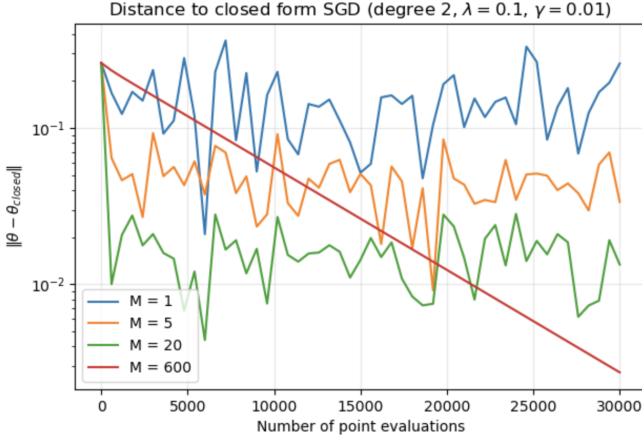


Figure 12: Distance between θ_{SGD} and $\theta_{closed-form}$ using stochastic gradinet descent with different bach sizes M . $n=600$ so $M=600$ is equal to full-batch GD. learning rate γ is constant and epochs=50.

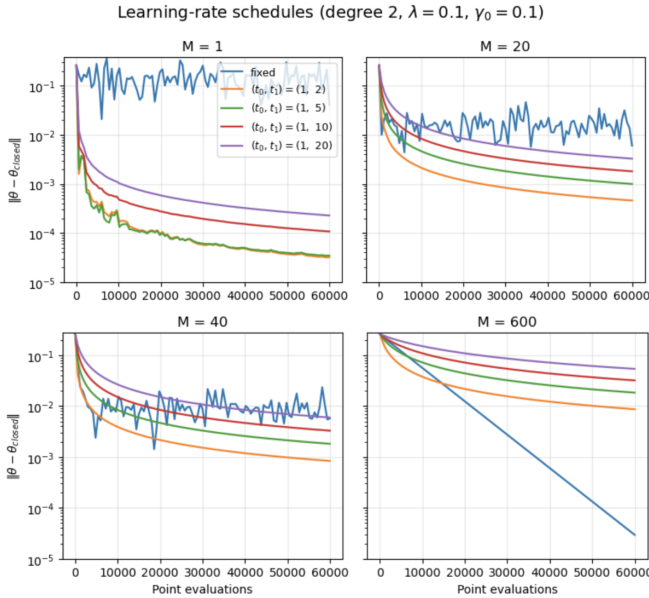


Figure 13: Distance between θ_{SGD} and $\theta_{closed-form}$ using stochastic gradinet descent with different bach sizes M and different schedules (t_0, t_1) . learning rate is no longer constant and epochs = 100 ($n = 600$ points).

rate with a schedule (t_0, t_1) , as shown in Figure 13. The fluctuations seen with a fixed learning rate disappear, and the error keeps decreasing. The downside is that the shrinking step size also slows the convergence. This is clearest for full-batch gradient descent ($M = 600$), which has no gradient noise to remove, so the decay only turns its steady exponential decrease into a much slower one. If we are limited to about 10 000 evaluations, the lowest error of the settings we tested is obtained with $M = 1$ and a schedule $(t_0, t_1) = (1, 2)$ or $(1, 5)$, at roughly 10^{-4} . After 50 000 evaluations, full-batch gradient descent with

a fixed learning rate has reached about the same order of error (10^{-4}), and its curve is still falling steadily while the SGD curves flatten out. For the final model selection we want as accurate parameters as possible and we do not have a limit on the number of evaluations. Full-batch gradient descent without a strictly decaying learning rate is therefore preferable for this problem.

E. Lasso regression

Figure 14 shows the Lasso cost as a function of the iteration ($\gamma = 0.01$, $\lambda = 0.01$). Lasso has no closed-form solution, so instead of the distance to the solution as in Figure 11, the y-axis shows the cost. When doing gradient descent with the Lasso cost, we need to choose a subgradient for the L1 penalty. The L1 penalty, $|\theta|$, is not differentiable at $\theta = 0$. Its subgradient is -1 for $\theta < 0$, 1 for $\theta > 0$, and any value in $[-1, 1]$ for $\theta = 0$. We chose 0 as the subgradient at $\theta = 0$, which corresponds to using $\text{sign}(\theta)$ as the subgradient. Since $\text{sign}(0) = 0$, and $0 \in [-1, 1]$, this is a valid choice of subgradient. For degree 2 and 4 all methods approach the same cost, with RMSProp and Adam the fastest and plain gradient descent and AdaGrad the slowest. Plain gradient descent diverges from degree 6, and momentum from degree 8. This is consistent with the stability limit of gradient descent, $\gamma < 2/\mu_{\max}$, where the largest Hessian eigenvalue $\mu_{\max}$ grows with the degree. Momentum tolerates a somewhat larger step, so it fails later. The adaptive methods scale the step for each parameter, so they stay stable at all degrees shown. RMSProp does however show occasional spikes in the cost, also late in the run, while Adam is smoother. The adaptive methods, and Adam in particular, are therefore the better choice for finding a Lasso model for the Runge function.

F. Estimate an optimal model

Putting everything together, Table I shows the final results. OLS and Ridge were fitted with the analytical solution, since it is fast and as accurate as possible. Lasso was fitted with full-batch gradient descent using Adam, as concluded earlier. The table shows the best models for degrees between 1 and 15 and λ between 10^{-6} and 1. The best OLS model has degree 10 with an average MSE of 0.01307. The best Ridge model also has degree 10 and $\lambda = 10^{-6}$, with an average MSE of 0.01255. The best Lasso model also has degree 10, but with $\lambda = 10^{-4}$ and an average MSE of 0.01624. Overall, Ridge is the most accurate, but only slightly better than OLS. Its best λ is the smallest on the grid, so the best Ridge model is almost equal to OLS. Since the folds are taken from all data points, these are cross-validation errors and not test errors.

For degrees 1 to 7 the three methods give almost the same error (differences below 0.001). Here the error is

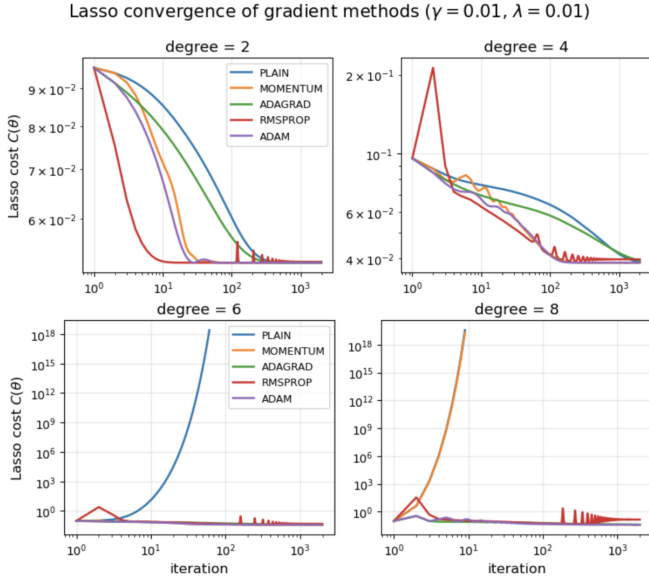


Figure 14: Lasso cost $C(\theta)$ as function of iteration using lasso regression with the 5 gradient descent methods. The different plots show different degrees of the model ($n = 60$ points).

dominated by bias, since a low-degree polynomial cannot follow the Runge function, and a penalty cannot help with this because it shrinks the parameters, which if anything adds bias. The methods separate only at high degree. From degree 11 the OLS error increases, to 0.0201 at degree 15, while Ridge stays between 0.0130 and 0.0137 and Lasso at about 0.017. We interpret this as the variance term. The cost function is a bowl that is very steep in some directions and almost flat in others. We know the bowl has such an ill conditioned shape from the eigenvalues of the hessian. For the degree 10 case the smallest and largest eigenvalues are $(\mu_{min}, \mu_{max}) = (2.96 * 10^{-7}, 2, 54)$. Therefore in a flat direction the cost barely changes when θ moves, so the data determine the position of the minimum poorly. A small change in the training data, such as different noise ($\sigma = 0.1$ in our case), moves the minimum a long way along the flat direction, and the fitted parameters and predictions vary a lot between data sets. This is high variance. The Ridge term adds 2λ to every eigenvalue of the Hessian, which lifts the flat directions and makes the minimum better determined. Since the flat directions have very small eigenvalues, a very small λ is enough, which is consistent with the best Ridge λ being 10^{-6} . The L_1 term of Lasso adds no curvature, so it does not lift the flat directions in the same way. It reduces the variance by pulling parameters towards zero instead.

For this reason, at degrees 14 and 15 OLS has the highest MSE. The variance grows with complexity, and at a fixed degree OLS has no way of reducing it, unlike the L_2 and L_1 penalties of Ridge and Lasso regression. These penalties add bias, but at high degree the reduction in

variance clearly outweighs it. Neither penalty can reduce the noise σ^2 . At degree 10, where all methods have their minimum MSE, Ridge is only slightly better than OLS and its best λ is the smallest on the grid, so there is little variance left to remove.

Lasso is the weakest model at its best degree. One possible reason is that our Lasso fit with Adam is not fully numerically optimal. The Hessian is ill-conditioned at high degree, so the 3000 iterations may not be enough to reach the minimum. Final parameter using Lasso also did not contain any 0s, so no features were excluded from the model. A better approach with the gradient descent may have caused the lasso parameters to come closer to a potentially better minimum than what we got.

Table I: Cross-validated MSE ($k = 5$) for each polynomial degree. For Ridge and Lasso the value is for the best λ on the grid, which is listed next to it. The lowest value for each method is in bold.

Degree	OLS	Ridge	λ_{Ridge}	Lasso	λ_{Lasso}
1	0.08726	0.08691	10^0	0.08659	10^{-1}
2	0.04508	0.04505	$10^{-1.5}$	0.04496	$10^{-1.5}$
3	0.04536	0.04522	$10^{-1.5}$	0.04495	$10^{-1.5}$
4	0.02869	0.02867	10^{-3}	0.02849	$10^{-2.5}$
5	0.02923	0.02895	$10^{-2.5}$	0.02862	$10^{-2.5}$
6	0.01760	0.01760	10^{-5}	0.01756	10^{-6}
7	0.01790	0.01783	10^{-4}	0.01765	$10^{-3.5}$
8	0.01467	0.01459	10^{-5}	0.01699	$10^{-5.5}$
9	0.01547	0.01489	10^{-5}	0.01707	10^{-6}
10	0.01307	0.01255	10^{-6}	0.01624	10^{-4}
11	0.01377	0.01263	10^{-6}	0.01630	10^{-4}
12	0.01527	0.01295	$10^{-5.5}$	0.01693	10^{-4}
13	0.01495	0.01311	$10^{-5.5}$	0.01704	$10^{-3.5}$
14	0.01987	0.01356	10^{-5}	0.01704	10^{-4}
15	0.02008	0.01373	10^{-5}	0.01727	10^{-4}

V. CONCLUSION

Our analysis of regression methods applied to the Runge function demonstrates the critical importance of model selection and regularization in managing the bias-variance tradeoff. Through cross-validation, we identified the optimal complexity for each method, ordinary least squares (OLS), Ridge regression and Lasso regression. All 3 methods point to the same model which was a 10 degree polynomial model.

OLS provided a baseline approach but suffered from high variance at higher polynomial degrees, failing to generalize well to unseen data. Ridge regression successfully reduced the magnitude of some parameters causing more bias but less variance, overall making the model less overfitted. Lasso regressions L_1 penalty also attacks the variance increase of higher degree model, however the optimal model still performed worse than both OLS and Ridge. This could be because of the limits of our gradient descent method that may have not fully diverged to

the optimal parameters.

Among the three methods, Ridge regression turned out to be the superior choice for this problem, demonstrating that some regularization can make model better suited for unseen data compared to the OLS. Its hard to say if Lasso regression could have potentially performed even better since we arent sure if our gradient descent ever reached the globalm minimum. But our results showed that it was the weakest of the 3 methods.

VI. APPENDIX

A. Use of AI/LLM Tools

A.1: Tools used. Claude Sonnet 5.5 (claude.ai, September–October 2026).

A.2: Level of LLM use for the text of the report.

Table II: Level of LLM use for the text of the report.

Section	Level
Abstract	1
Introduction	2
Theory	2
Methods	0
Results/Discussion	2
Conclusion	2

A.3: Level of LLM use for the code.

Table III: Level of LLM use for the code, by project part.

Part	1a, 1b	1c, 1d	1e, 1f	1g, 1h	1i
Level	2	2	2	2	2

-
- [1] C. Runge, *Zeitschrift für Mathematik und Physik* **46**, 224 (1901).
 - [2] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, “JAX: composable transformations of python+numpy programs,” (2018).
 - [3] e. F. Pedregosa, *Journal of Machine Learning Research* (JMLR) (2011).
 - [4] M. Hjorth-Jensen, “Applied data analysis and machine learning: Lecture notes,” <https://github.com/CompPhysics/MachineLearning> (2026).
 - [5] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. (Springer, New York, 2009).